UNIVERSIDAD NACIONAL DE LOJA

FACULTAD ENERGÍA, LAS INDUSTRIAS Y LOS RECURSOS NATURALES NO RENOVABLES

CARRERA DE COMPUTACIÓN

ESTUDIANTE: Marco Vinicio Orozco Guamán

FECHA: 11/11/2026

MODALIDAD: Presencial

CICLO: Quinto

PARALELO: "A"

MATERIA: Sistemas Digitales

DOCENTE: Ing. Luis Darío Sinche Cueva

TEMA:

Apuntes en Clase

PERIODO ACADÉMICO

SEPTIEMBRE - MARZO 2027

Clase 1

- Un sistema embebido es un sistema que ejecuta una sola tarea
- Un microcontrolador es una computadora con una fracción de cpu memoria periféricos por dentro de un solo chip
- Un microprocesador solo es una cpu lo demás está aparte y está conectado

En este caso el ser un chip de propósito único todo el chip tiene cpu memoria y periféricos en un microchip, pero su velocidad es poca y no podría aguantar lo que un microcontrolador sí, el conjunto masivo de transistores, da el lugar a las compuertas y su conjunto forman las ALUS fingiendo como si fueran cerebros lógicos, y para el consumo dependerá del fabricante al igual que los valores uno y cero lógico dependerán de lo mismo.

Clase 2

La diferencia de un procesador y un microprocesador no existe, ya que son lo mismo

Los microcontroladores están presentes en sistemas embebidos ósea que solo cumplen un solo propósito y dentro de él está todo unificado ósea no necesita de periféricos extras ya que contiene: Cpu, Buses, Memoria y periféricos, todo esto está en conjunto.

Existe 3 tipos de buses que sirven para transportar información en el micro controlador:
1. Bus de dirección: Nos dice a dónde va la informacion

2. Bus de datos: Que información está llevando

3. Bus de control: Que operación se ara

Un núcleo es una CPUQue puede funcionar en paralelo y lo que con tiene la CPU es:
1. Contador de programa: Nos proporciona la dirección de la siguiente instrucción.

2. Unidad de control: Lee la instrucción y decide que es lo que va ah realizar

3. ALU: Se encarga de las operaciones lógicas.

4. Registros: Son casillero donde se almacena la información.

Clase 3

Leer:

- CPU pore 0x25 en direcciones
- Activa “Leer” en el bus D
- La memoria pone la info en el bus
- CPU captura el Byte

Escribir:

- CPU pore 0x25 en direcciones
- Pore 001000 en el bus de datos

- Activa “escribir”
- Reemplaza el byte

Arquitectura de Von Neuran: Un solo bus lleva y trae la instrucción y el dato ordenadamente

Arquitectura hardbard: 2 buses un es para los datos y otro para la memoria

Memoria flas: Es no volátil y se escribe al subir un programa en un JPE Ejemplo: un libro

Sram: Es volátil y guarda variables mientras corre el programa Ejemplo: una pizarra

Eeprom: Es no volátil y es delicada y dispone de pocos datos del programa Ejemplo: Una Libreta

1Byte 8 bits	1KB 1024 bytes	1MB 1024 kb		Atmega 328P	ESP32
			FLASH	32kb	4Mb
			SRAM	2kb	520kb
			EEPROM	1kb	Se emula en flash

Todas las memorias tienen una vida útil:

FLASH: 10000 ciclos garantizados

SRAM: No sufre por ser volátil

EEPROM: 100000 ciclos, un ciclo es leer y otro escribir

En EEPROM no se escriben lopps.

Periféricos:

GPIO: Pone el pin en 1 o 0 o lo lee.

Bits, operaciones bit a bit y configuración in / out:

El bit mas significativo es el que tiene más información

Un alto es 1 y bajo 0, V es 5v Arduino y 3.3v Esp32 un bajo es el alto en intervalo abierto, los altos y bajos tienen rangos antes y después de la zona de incertidumbre.

10000001 = 81 en decimal y 0x81 Hexadesimal

0xFF = 255 en decimal y 11111111 en binario

0x08 = 00001000 en binario 8 en decimal

200 = 0xC8 en exadecimal y 11001000 en decimal

Mascara: 1 << n señala un bit

Poner bit en 1	Reg l=(1<<n)

Poner bit en 0	Reg &=~(1<<n)
Poner bit n invertido	Reg ^=(1<<n)
Preguntar por bit n	If Reg & (1<<n)

Clase 4

Reg l=(1<<0); Reg &=~(1<<7); Reg ^= 0.0F5; Reg = 1010 1010

RISC y CISC: Biblioteca de instrucciones microcontrolador

CISC es mas completa que RISC

LDI: Carga inmediatamente

APP: Suma bit a con bit B, guarda en bit a

SBI: Registra 1 en bit a: 0x05,5 > posición

CBI: Registra 0 en bit a: 0x05,5

RJMP: Salto de línea adelante o tras

Clase 5

DDRB: dirección Entrada y Salida, si el bit es 1, es Salida.

PORTB: en combinación con DDRB:

DDRB = 1 — salida	DDRB = 0 — entrada
Portb = 0 → 0V	Portb = 0 Alta impedancia
Portb = 1 → 5V	Portb = 1 R. pullup, siempre lee 1 lógico, y con un pulsador da el 0 lógico

Análogos:

Sensores (frágiles).

Puerta de vidrio.

Pinb: leer voltaje real, 1 alto 0 bajo.

¿0x81 en Binario?

81 = 0100 0001

Línea que apague el bit 4 port b

PORTB &= ~(1 << 4); o CBI 0x08, 4

Con DDRD2 = 0 y PORTD2 = 1, ¿Qué lee el pinD2?

Lee 0 porque está en modo con resistencias pull-up.

Clase 6

Características ESP-32

Ancho de palabras de 32 bits

2 Nucleos Xtensa Lx6, hasta 240 MHz

SRAM de 520 KB

Flash 4 MB, Chip a parte del módulo. SPI

Tensión en 3.3 V

Pin y Periféricos configurable, respecto a su comunicación

Alimentación → Regulador → Modulo ESP-32

SV + Datos

SV a 3,3 V
Chip Regulador
USB-Serial
Warp

* Chip 2 núcleos
* Flash 4 MB
* Antena wifi Bluetooh

- Comunicación Serial Warp: Traductor de comunicación USB a Serial Warp para que el ESP-32 lo entienda.
- Pin: Entrada Física que actúa como frente ente lo de afuera y el microcontrolador.
- Periférico: Circuito que cumple una función, liberando a la CPU de tareas de específicas.

Entre los pines y periféricos existe un GPIO Matrix, y un multiplexor de entradas y salidas, conectándolos entre sí, a esto se le llama entrada y salida de propósito general en una matriz.

- Dirección entrada y salida: GPIO-EN ABLE-REG
- Escribir Salida: GPIO-OUT-WITS-REG II GPIO-OUT-WITS-REG
- Leer entrada: GPIO-IN-REG
- Pull up externa: registro {O_MUX de cada pin.

WITC y WITS: Limpiar y setear 1 en algo en 0

Sistemas digitales – Aprendizaje- Buscando En errores

W1TS y W1TS son registros seguros que hacen fácil las operaciones multihilos, escrbiendo y en bits del W1TS o W1TC.

Capas de abstracción

Mientras mas alto sea el nivel del lenguaje mas lenta y poco controlada es le ejecución.

Código → ESP-32 – ESP-IDF-Registro- hardware

Al encenderse el microcontrolador revisa los pines de strapping.

GPIOO: en bajo→ modo de carga, botón BOOT

GPIO2: debe estar en bajo o alare para cargar. LEO azul.

GPIO12: en alto→ flash a 1,8 V no arranca.

Para GPIO5 y GPIO15 se leen mejor no conectar nada ahí al arrancar.

Los bits de 1 de las máscaras también tienen tipo.

Para constantes (ESP-32 las guarda en la flash)

#define LED 16 [no tipo, solo por compatibilidad] Texto.

const uint8_t LED = 16; [Fijas].

constexpr uint8_t LED = 16; [Calculadas al copiar].

enum MODO {Lento, Rapido};

Volatil: El valor inicial puede cambiar fuera del flujo (interrupción).

Static: El valor no se vuelve a encerar.

delay(int): pausar programa.

millis(): contar tiempo transcurrido.

Clase 7

Comunicación UART: Capa de datos y despliegue en serie, USB. Usa comunicación en paralelo.

VIN: es entrada de voltaje.

3V3: es salida para alimentar.

En la ROM están las configuraciones de los pines de strapping.

Clase 8

Temporizadores de hadware

Recoder: promoción, cuando los bits resultantes es mayor al espacio, se desborda.

Periférico: circuito dentro del microcontrolador.

Contador: registro del temporizador, unidad de memoria exclusiva del temporizador, la CPU solo le da instrucciones y lo deja ir de largo hasta completar el tiempo.

La CPU:

1. Configura.

2. Habilita: Bit de alimentación (1).

3. Manda a contar al temporizador.

4. Si acaba, lo detiene.

Reloj: sube y baja señal a ritmo fijo, es una señal.

APB: Bus de periférico avanzado, con reloj de 80 MHz.

Preescaler: Divisor antes del contador, contar menos pulsos según flancos de subida, contador avanza N veces más lento.

Resolución: lo que dura un pulso = 1/f mismo que el periodo

La alarma se da después del tiempo dado, es una variable guardada en un registro, es como el contador entonces el contador está preguntando a la alarma si es el mismo tiempo.

Tiempo = alarma xresolucion

Alarma = tiempo*f

F=1000000 por segundo o 1MHz

Tiempo deseado 0.5s

1. Calcula la resolución

$$R = \frac{1}{t} = \frac{1}{1000000} = 1\mu s$$

2. Calcular la alarma

A=0,5*1000000=500000 ticks suena la alarma

Comprobar

500000*1μs=0,5

A*r=t

Prescaler

$$N=\frac{80000000}{1000000} = 80$$

Tengo 10 000 ticks por segundo y quiero alarma a los 2500 ticks

1 calcular resolución

$$R = \frac{1}{f} = \frac{1}{10000} = 100\mu s \ o \ 1ms$$

2 calcular tiempo

t=2500*0,0001=0,25s

3 prescaler

$$N=\frac{80000000}{10000} = 8000$$

Autorecarga: Al alcanzar la alarma el contador vuelve a 0

Desbordamiento: Máximo de bits, el contador se reinicia a la fuerza, alcanzó límite físico, no llegó a la alarma

La ESP32 tiene 4 temporizadores en 2 grupos TMG0 y TMG1, cada uno con timer0 y timer1

Core: paquete de programación para Arduino ID con las especificaciones y trabajar con la ESP32

ISR: función que se ejecuta cuando hay una interrupción, rutina de Servicio de Interrupción

Callback: función que se da a otro para ser llamado

IRAM: instrucciones de la RAM de forma rápida

DRAM: datos de RAM

Ambos casos no van a la flash

Patrón bandera: ISR avisa para que loop() trabaje

Clase 9

Sondeo=polliy (si el loop ya pauso)

Interrupción externa: es por un aviso del chip mediante un pin.

IRS: interrp service Routive: avisado ejecuta la función.

Watch dog: reinicia el chip si algo se queda trabado mucho tiempo carga desde la flash para interrupciones largas.

DIF sondeo e intempcin: revisa si ya llegó puede variar si se está ejecutando instrucciones. Intempcing pausa el loop lo hace el periférico GPIO asignado este le avisa a la CPU.

Nivel: tiempo que dura en un estado

Flanco: el instante que cambia de estado

Falling: flanco de bajada

Rising: flanco de subida

Change: toma en cuenta los dos cambios

IRAM: instrucciones que copia para trabajar

DRAM: guarda datos para trabajar

IRAM_ATTR: guarda esa función en la IRAM.

Compilador es ahorrativo, si una varía no cambia después de un tiempo la manda a la flash y guarda una copia, por lo que si cambia solo es en Flash, en el código sí cambió pero él hardware no sabe que cambió.

Volatile: hace que la variable no se pierda se para cuando la ISR y el loop tocan la misma variable.